

RIK JANSEN

Senior Software Engineer

rik.jansen.ict@outlook.com | linkedin.com/in/rik-jansen-b24a463a1/ | Roden, Drenthe, Netherlands | +31 685549850

SUMMARY

Senior Python React Full Stack Engineer with experience across fintech, healthcare, SaaS, logistics, and enterprise workflow platforms, building secure products with Python 3.8–3.12, Django 3.x–5.x, FastAPI 0.95+, React 16.x–18.x, TypeScript 4.x–5.x, PostgreSQL, Redis, AWS, Docker, and CI/CD. Skilled in modernizing legacy systems, migrating monolith features into API-driven services, fixing production bugs, improving frontend performance, and delivering user-focused features that reduce operational effort, increase reliability, and support scalable business growth. Strong background in REST API design, authentication, testing, observability, and Agile delivery for remote international teams.

SKILLS

- **Languages:** Python 3.8–3.12, JavaScript ES6+, **TypeScript 4.x–5.x**, SQL, HTML5, CSS3
- **Frontend:** **React 16.x–18.x**, Redux, React Router, React Hook Form, Material UI, Bootstrap, Responsive UI
- **Backend:** **Django 3.x–5.x**, Django REST Framework, **FastAPI 0.95+**, Flask, REST APIs, WebSocket APIs
- **Database:** **PostgreSQL 10–16**, MySQL, Redis 5–7, Query Optimization, Indexing, Transactions
- **Cloud & DevOps:** AWS ECS, EC2, S3, RDS, Lambda, CloudWatch, Docker, GitHub Actions, Jenkins, Nginx
- **Testing:** Pytest, Django TestCase, Jest, React Testing Library, API Integration Testing, Regression Testing
- **Architecture:** Microservices, Monolith Migration, API Design, Authentication, RBAC, Caching, Async Jobs, CI/CD

EXPERIENCE

Senior Software Engineer

Viario Networks Inc.

June 2021 - October 2025

- Built secure **Python 3.8–3.12** and **React 17.x–18.x** full stack modules for fintech and SaaS platforms, connecting Django services, FastAPI endpoints, TypeScript components, and PostgreSQL workflows into one maintainable product architecture.
- Implemented a new customer onboarding dashboard with **React 18**, **TypeScript 5.x**, role-based forms, API-driven validation, and reusable components that reduced manual account review time by **32%**.
- Designed **FastAPI 0.95+** services for payment status, audit events, and internal reporting, using Pydantic schemas, async endpoints, Redis caching, and structured logging to support high-volume business operations.
- Fixed a production authorization bug where cached permissions were not refreshed after role changes, solving it with token invalidation, Redis key scoping, integration tests, and stricter **Django 4.x** middleware checks.
- Migrated legacy **Django 3.x** modules into **Django 4.x–5.x** compatible code by replacing deprecated settings, upgrading authentication flows, refactoring ORM queries, and stabilizing CI regression coverage.
- Improved React page performance by splitting large bundles, memoizing expensive dashboard widgets, removing redundant API calls, and applying lazy loading that improved average page load speed by **38%**.

- Built internal admin features for billing reconciliation, user lifecycle management, and document review, combining **Python**, React, PostgreSQL transactions, and Celery background jobs for reliable operational workflows.
- Resolved a recurring invoice duplication issue caused by retrying webhook callbacks by adding idempotency keys, unique database constraints, queue retry limits, and alerting around failed payment events.
- Created REST and WebSocket integrations between frontend dashboards and backend event services, giving operations teams near real-time visibility into user activity, system status, and exception trends.
- Strengthened deployment reliability with **Docker**, **GitHub Actions**, AWS ECS, RDS, S3, and CloudWatch alarms, helping the team reduce failed releases by **27%** through automated checks and safer rollbacks.
- Collaborated with product managers, QA engineers, and designers to convert ambiguous customer workflow problems into scoped features, API contracts, UI states, and measurable delivery milestones.
- Mentored engineers on **Python type hints**, React component boundaries, API testing, code review discipline, and production debugging habits that improved team consistency across distributed development squads.

Software Engineer

ArnAmy, Inc.

August 2017 - May 2021

- Developed healthcare and logistics web applications with **Python 3.6–3.9**, **Django 2.2–3.2**, Flask 1.x, **React 16.x–17.x**, PostgreSQL, Redis, Celery, and AWS services for multi-client business platforms.
 - Built a patient intake and case tracking interface with **React 16.x**, Redux, controlled forms, backend validation, and Django REST endpoints that reduced coordinator data entry effort by **29%**.
 - Implemented secure authentication flows with JWT, session hardening, password reset logic, role-based access, and audit logging to protect sensitive records across healthcare workflow modules.
 - Fixed a timezone-related scheduling defect where appointment reminders were sent at the wrong local time, solving it with UTC storage, timezone-aware Django utilities, regression tests, and clearer frontend formatting.
 - Migrated older **Python 2.7** utility scripts and mixed Django views into **Python 3.7+** services, replacing byte-string assumptions, updating dependencies, and improving automated test coverage.
 - Created reusable React table, filter, modal, and form patterns that allowed product teams to ship reporting screens faster while keeping validation and API error handling consistent.
 - Optimized slow reporting endpoints by analyzing PostgreSQL query plans, adding selective indexes, replacing nested loops, and caching stable reference data to improve report generation speed by **41%**.
 - Integrated third-party APIs for shipping, email notifications, and customer verification, adding retry logic, timeout handling, and error dashboards to make external-service failures easier to diagnose.
 - Improved deployment quality with **Docker**, Jenkins, AWS EC2, RDS, S3, and environment-specific configuration, reducing environment mismatch issues by **24%** across staging and production.
 - Supported Agile delivery by breaking large features into backend contracts, frontend states, QA scenarios, and release notes that gave stakeholders clearer visibility into delivery risk.
 - Used **Pytest**, Django TestCase, Jest, and React Testing Library to cover critical authentication, scheduling, and reporting behavior before releases reached customer-facing production environments.
-

Software Developer

Webiz Digital

December 2015 - July 2017

- Built business websites, internal portals, and early SaaS tools using **Python 3.5–3.6**, **Django 1.10–1.11**, Flask 0.12, React 15.x, Redux 3.x, PostgreSQL 9.x, and Bootstrap-based interfaces.
- Developed a React 15.x customer portal for profile updates, service requests, and document uploads, connecting UI components to Django REST endpoints and improving request completion efficiency by **26%**.
- Implemented backend validation, file upload handling, email notifications, and admin review queues in **Django**, helping non-technical operations users process client requests with fewer manual checks.
- Fixed a serious form submission bug caused by duplicate browser refreshes by adding CSRF-safe server validation, database uniqueness rules, and frontend disabled states during async requests.
- Migrated several legacy jQuery-heavy pages into React 15.x components gradually, preserving existing Django templates while introducing reusable UI patterns and cleaner client-side state handling.
- Improved API response time by refactoring ORM queries, adding pagination, reducing template-side database calls, and caching stable lookup values, which improved average listing speed by **34%**.
- Created scheduled Python jobs for CSV imports, email queue processing, and nightly data cleanup, adding logs and failure notifications so support teams could resolve issues before clients reported them.
- Collaborated with designers to convert static mockups into responsive Bootstrap and React interfaces, balancing browser compatibility, accessibility basics, and maintainable frontend structure.
- Supported AWS EC2, S3, RDS, Nginx, Gunicorn, and Ubuntu deployments by preparing release checklists, fixing configuration issues, and documenting environment-specific setup steps.
- Participated in code reviews, bug triage, and client feedback sessions, using **Python** and JavaScript flexibly across frontend fixes, backend features, and production support tasks.

Junior Software Engineer

High Touch Technologies

August 2014 - November 2015

- Built and maintained internal business applications with **Python 2.7–3.4**, **Django 1.7–1.8**, Flask 0.10, JavaScript, jQuery, Bootstrap, MySQL, and PostgreSQL for service and operations teams.
- Created data entry screens, reporting pages, and admin tools that helped staff replace spreadsheet-heavy workflows with browser-based forms and searchable operational records.
- Fixed validation and save errors caused by inconsistent nullable fields by tightening Django model constraints, adding form-level checks, and documenting expected data formats for support users.
- Implemented small backend features for customer records, ticket tracking, and export workflows, using Django views, templates, SQL queries, and Python scripts to support daily office processes.
- Improved a slow monthly report by simplifying joins, adding database indexes, and moving repeated calculations into reusable Python helpers, reducing preparation time by **22%**.
- Supported migration work from older server scripts into cleaner Django modules by separating view logic, model queries, and template rendering into easier-to-maintain application layers.
- Worked with senior engineers to debug production issues through log review, database inspection, browser testing, and step-by-step reproduction of user-reported application problems.
- Maintained frontend screens with Bootstrap, jQuery, HTML, and CSS while learning component-based UI practices that later supported stronger **React** development in full stack projects.
- Prepared release notes, test cases, and basic deployment documentation, helping the team reduce repeated setup questions by **18%** during maintenance releases and onboarding.

EDUCATION

Bachelor's degree in Computer Science
University of Amsterdam
2010 - 2014